

Python Code

```

1  # #####
#####
2  # coins = [1, 2, 5]
3  # T = 5
4  # Number of ways = 4
5
6  # Ways:
7  # 1) 1+1+1+1+1
8  # 2) 1+1+1+2
9  # 3) 1+2+2
10 # 4) 5
11
12 #Task 1 – Naive Recursive Approach (Brute Force)
13
14 def count_ways(coins,i,T):
15     if T == 0:
16         return 1
17     if T < 0:
18         return 0
19     if i == len(coins):
20         return 0
21     include = count_ways(coins,i,T-coins[i])
22     exclude = count_ways(coins,i + 1,T)
23     return include + exclude
24 def collect(coins, i, T, path, result):
25     if T == 0:
26         result.append(list(path))
27         return
28     if T < 0 or i == len(coins):
29         return
30     collect(coins, i, T -coins[i],path+[coins[i]], result)
31     collect(coins, i + 1, T, path,result)
32
33 def print_ways(result):
34     print("Ways:")
35     for i in result:
36         for j in i:
37             print(j,end = " ")
38         print()
39 coins = [1, 2, 5]
40 T = 5
41 ans1 = count_ways(coins, 0,T)
42 print("___Task 1 answer:___")
43 print("Numbers of ways ",ans1)
44 result1 = []
45 collect(coins, 0, T, [], result1)
46 print_ways(result1)
47 #####
#####
48 #Task 2 – Top-Down DP with Memoization
49 def count_ways(coins, i, T, memo):
50     if T == 0:

```

```

51         return 1
52     if T < 0:
53         return 0
54     if i == len(coins):
55         return 0
56     if (i, T) in memo:
57         return memo[(i, T)]
58     include = count_ways(coins, i, T - coins[i], memo)
59     exclude = count_ways(coins, i + 1, T, memo)
60     memo[(i, T)] = include + exclude
61     return memo[(i, T)]
62 coins = [1, 2, 5]
63 T = 5
64 memo = {}
65 ans2 = count_ways(coins, 0, T, memo)
66 print("__Task 2 answer:___")
67 print("Numbers of ways ", ans2)
68
69 #####
#####
70 #Task 3 – Bottom-Up Dynamic Programming
71 def count_ways(coins, T):
72     n = len(coins)
73     dp = [[0] * (T + 1) for _ in range(n + 1)]
74     for i in range(n + 1):
75         dp[i][0] = 1
76         for i in range(n - 1, -1, -1):
77             for t in range(1, T + 1):
78                 include = dp[i][t - coins[i]] if t - coins[i] >= 0 else 0
79                 exclude = dp[i + 1][t]
80                 dp[i][t] = include + exclude
81     return dp[0][T]
82 coins = [1, 2, 5]
83 T = 5
84 ans3 = count_ways(coins, T)
85 print("__Task 3 answer:___")
86 print("Numbers of ways ", ans3)
87
88
89 #####
#####
90 #Task 4 – Limited Coin Supply
91 def count_ways(coins, counts, i, T, memo):
92     if T == 0:
93         return 1
94     if i == len(coins):
95         return 0
96     if (i, T) in memo:
97         return memo[(i, T)]
98     ways = 0
99     for k in range(counts[i] + 1):
100         if k * coins[i] > T:
101             break
102         ways += count_ways(coins, counts, i + 1, T - k * coins[i], memo)
103     memo[(i, T)] = ways

```

```
104     return ways
105 coins = [1, 2]
106 counts = [3, 1]
107 T = 4
108 memo = {}
109 ans4 = count_ways(coins, counts, 0, T, memo)
110 print("_____Task 4 answer:_____")
111 print("Numbers of ways ", ans4)
112
113 #####
#####
114
```